Building an AI-powered solution to uncover true research impact

The Jackson Laboratory, Bar Harbor, Maine (jax.org), is a crucial resource for biomedical research. JAX maintains and distributes more than 13,000 genetically defined mouse strains to thousands of scientific research organizations around the world.

When these animals are successfully used in research, the resulting discoveries are typically documented in peer-reviewed scientific publications. The Jackson Laboratory is a 97 year-old non-profit science institute and has long-established standards for how its biomaterials should be identified and credited in those publications.

There is, however, a fundamental problem. Compliance with those standards is voluntary.

Consequently, the scientific literature contains considerable variation in how JAX mouse strains are identified.

That creates a surprisingly difficult business problem.

JAX cannot reliably identify every publication resulting from research involving its animals. Nor can it always connect a published research result back to the JAX strain that was used—or, ultimately, to the organization that purchased it.

The result is that some of JAX's most valuable scientific successes can remain effectively invisible.

The laboratory may have supplied the mouse that enabled an important cancer discovery, but if the publication doesn't use the expected identifier, that contribution can be extremely difficult to find.

In other words, JAX may have helped create a scientific "win" without being able to reliably notch the win.

And those wins matter. They provide evidence of scientific impact, help researchers discover relevant JAX strains, support relationships with research institutions, and potentially create new opportunities for sales and collaboration. Revenue from the genetically defined mouse strains, in turn, helps support two additional JAX missions: in-house research and advanced-level education.

While employed at The Jackson Laboratory in 2023 as Senior Manager of Digital Enablement, I recognized an opportunity to address this problem using automated text analysis, cloud computing, and artificial intelligence.

The basic idea was simple: **Instead of waiting for researchers to identify JAX correctly, search the scientific literature for the evidence directly.**

The identifier problem

The first challenge was understanding how many different ways a JAX mouse could be identified in a scientific publication.

I identified at least seven categories of identifiers and naming conventions that could potentially point back to a specific JAX strain.

1. RRID

The Research Resource Identification number is the JAX gold-standard identifier.

For example:

`RRID:IMSR_JAX:000664`

RRIDs are specifically intended to make research resources easier to identify and track across scientific publications.

2. MGI accession identifier

More generally, Mouse Genome Informatics (MGI) maintains the authoritative genetic nomenclature and associated information for mouse strains.

An MGI identifier can appear in the scientific literature in a format such as:

`MGI:NNNNNN`

3. JAX stock number

A JAX stock number is another potentially valuable clue.

For example:

`000664`

The problem is that a six-digit number appearing in a scientific paper isn't necessarily a JAX stock number. Context matters.

4. Full strain name

A publication may use the complete strain designation:

`C57BL/6J-Dnm2b2b2159Clo/J`

This is highly informative but considerably less likely to appear consistently than a shorter name.

5. Strain short name

Authors may use only the common strain designation:

`C57BL/6J`

This is much easier to encounter—but much harder to attribute uniquely to JAX.

6. Synonym

A publication may use a synonym which could be a variation on any strain name designation.

For example:

C57BL/6J-b2b2159Clo/J

7. Common name

Finally, researchers may use an informal or commonly accepted name:

B6

B6J

B6/J

The problem becomes immediately apparent.

A search for:

RRID:IMSR_JAX:000664

is relatively straightforward.

A search for:

C57BL/6J

is considerably more complicated.

The latter designation can appear in thousands of scientific publications without any explicit indication that the animals were obtained from JAX. That distinction became central to the architecture I envisioned.

The source of the data

Fortunately, an extraordinary resource already existed.

The National Institutes of Health's National Library of Medicine (NIH/NLM) makes the PubMed Central (PMC) Article Dataset available as full-text scientific literature. The dataset includes biomedical and life-sciences journal articles archived in PMC and made available for text mining and other forms of secondary analysis subject to the applicable licensing terms.

The PMC article datasets are hosted in Amazon S3.

This created an intriguing possibility.

Rather than building a system that searched the public web one article at a time, I could work directly against a large, structured corpus of scientific literature in the AWS ecosystem.

One example of a publication referencing a JAX strain is:

Kapur, M., Ganguly, A., Nagy, G., et al.
"Expression of the Neuronal tRNA n-Tr20 Regulates Synaptic Transmission and Seizure Susceptibility."
Neuron, 2020; 108(1):193–208.e9.
PMID: 32853550
PMCID: PMC7572898
DOI: 10.1016/j.neuron.2020.07.023

The question was no longer:

> "Can I find this particular paper?"

It became:

> "Can I systematically examine millions of scientific articles for evidence that JAX animals were used in research?"

The architecture

The solution I designed centered around a pipeline with progressive stages of increasing specificity:

JAX Master Data > Identifier & Alias Lexicon

PMC Scientific Literature > Article Text Extraction

Exact / Pattern Matching > Candidate JAX References

Context Extraction > AI-Assisted Interpretation

Confidence Scoring >

S3 Research Evidence > AWS OpenSearch

Outcome: Research & Business Intelligence

The architectural plan did not call for "asking LLM to search millions of papers". Instead, I envisioned AI as one component in a larger information-processing pipeline.

---

1. Build a comprehensive JAX identifier lexicon

The first component was a machine-readable lexicon of JAX identifiers and their variants.

I envisioned querying Mouse Genome Informatics and related JAX reference information to construct a master table in which multiple names could resolve to one canonical JAX strain.

For example, each synonym would map to the canonical JAX strain identifier RRID:IMSR_JAX:000664

JAX 000664
000664
C57BL/6J
B6J
B6/J

A significant range of additional metadata values would be captured so that the lexicon would contain considerably more than a list of strings.

- canonical JAX stock number;
- RRID;
- MGI identifier;
- official strain name;
- short name;
- synonyms;
- common names;
- alternative spellings;
- search patterns;
- source URL;
- identifier type;
- confidence or specificity rating;
- effective date/version.

Not all identifiers are equally useful since some might prove highly specific and other types could produce false matches. The cross-referencing of identifiers was a critically important feature, making the lexicon much more than a dictionary. All this reference information became a knowledge model for AI to interpret further evidence.

---

2. Ingest the scientific literature

The second component was the scientific literature itself.

The NIH/NLM PMC Article Dataset provides a continuously updated corpus of full-text scientific publications, centralized at one Amazon S3 location. Rather than designing a limited-run process, I made the pipeline repeatable. The processing pipeline could periodically re-execute for the identification of new publications. In each case, the same retrieve the appropriate article representation, and extract the textual content and bibliographic metadata.

Conceptually:

PMC S3 Dataset


Article inventory
▼
New / changed articles
▼
XML / text extraction
▼
Text normalization
▼
JAX detection

Therefore, the pipeline was not actually "scraping the Internet." Instead, it was processing a structured scientific corpus in a way that could be much more repeatable, auditable, and scalable.

Important caveats: The authoritative source is PMC's current AWS access guide, together with the live bucket README.txt. These are particularly important because PMC completed a major change to its article-distribution system on August 26, 2026. Legacy dataset files were removed, so code based on older PMC directory structures may no longer work.

---

3. Find candidate references

The first search stage should be deliberately simple.

If the system knows that:

RRID:IMSR_JAX:000664

is an identifier of interest, it should search for it.

Likewise:

000664

and:

C57BL/6J

and all of their known aliases and patterns.

This is where conventional information-retrieval technology is extremely effective.

There is little reason to spend an expensive LLM call asking:

> "Does this article contain the exact string C57BL/6J?"

A conventional search engine can answer that question faster, more cheaply, and more deterministically.

The objective of this stage is therefore **candidate generation**.

It produces a collection of potentially relevant passages.

For every match, the system can retain:

- publication identifier;
- author information;
- journal;
- publication date;
- DOI;
- matched string;
- identifier type;
- location within the article;
- surrounding text;
- article section;
- source document.

For example:


```
{
  "pmcid": "PMC7572898",
  "pmid": "32853550",
  "matched_string": "C57BL/6J",
  "identifier_type": "strain_shortname",
  "context": "...C57BL/6J mice were used...",
  "journal": "Neuron",
  "publication_year": 2020
}
```


At this point, however, we have only found **evidence**.

We haven't established what that evidence means.

---

4. Add context

This is where the problem becomes much more interesting.

Suppose the system finds:

`C57BL/6J`

inside a publication.

That alone does not prove that the authors obtained animals from JAX.

The phrase might describe:

- an experimental mouse;
- a genetic background;
- a comparison group;
- a strain mentioned in the introduction;
- a strain described in another publication;
- or simply background scientific information.

The surrounding text therefore becomes critical.

Instead of storing only:

C57BL/6J

the system should retain a meaningful context window:

...Female C57BL/6J mice were obtained from
The Jackson Laboratory and maintained under
standard laboratory conditions...

That additional evidence dramatically changes the interpretation.

Now the system has a strong indication that the JAX strain was actually used experimentally.

---

5. Use AI to interpret the evidence

This is where I see the greatest opportunity for AI.

The LLM should not replace exact search.

It should **interpret the results of exact search**.

The AI can examine the matching passage and answer questions such as:

- Is this actually a reference to an experimental animal?
- Does the passage indicate that the animals were obtained from JAX?
- Is the strain name being used as a genetic background?
- Is the JAX stock number explicitly identified?
- Does another identifier in the same passage corroborate the match?
- Is this likely to be a false positive?
- Which canonical JAX strain does this evidence correspond to?

The model can then produce a structured interpretation.

For example:

Matched identifier:

C57BL/6J

Candidate JAX strain:
000664

Evidence:
"C57BL/6J mice were obtained from The Jackson Laboratory..."

Interpretation:
Experimental animals were obtained from JAX.

Confidence:
High

This is fundamentally different from asking an LLM to search an enormous document collection blindly.

The search engine finds the evidence.

The AI interprets the evidence.

Exact search + semantic search

There is another reason I became interested in combining conventional search with AI.

Scientific authors are inconsistent.

One researcher might write:

```
RRID:IMSR_JAX:000664
```

Another might write:

```
JAX 000664
```

Another:

```
C57BL/6J
```

Another might describe the animals in language that does not contain the canonical JAX identifier at all.

This creates two complementary search problems.

Exact search

Exact and keyword search is excellent for:

- RRIDs;

- MGI identifiers;
- JAX stock numbers;
- official strain names;
- known synonyms.

Amazon OpenSearch provides BM25-based keyword matching that is well suited to this type of retrieval.

Semantic search

Vector embeddings provide another capability.

Instead of looking only for an exact character sequence, semantic search can identify passages whose meaning is related to the query.

This is useful when the terminology varies.

This creates a useful division of labor:

Keyword search finds precise identifiers.

Semantic search finds conceptually related evidence.

AI interprets the evidence.

Automating the pipeline with agentic orchestration

For this solution to operate at scale, the pipeline should not depend on a person manually initiating each search, reviewing every match, or deciding what to do with ambiguous results. The process can instead be implemented as an orchestrated workflow in which deterministic automation handles the repetitive work. When interpretation or adaptive decision-making is required, the AI agent is invoked to evaluate the evidence and determine what additional processing is called for.

A scheduled orchestration layer can monitor the PMC corpus for new or changed publications, initiate ingestion, invoke identifier matching, and record processing status. Once candidate JAX references are discovered, an AI agent can evaluate the evidence and determine what additional processing is appropriate.

For example, an agent receiving a low-confidence match for C57BL/6J might:

examine the surrounding passage;

search the same publication for additional JAX identifiers;

look for references to The Jackson Laboratory;

compare the terminology against the JAX identifier lexicon;

retrieve additional context from the article;

evaluate whether the evidence indicates actual experimental use;

assign a confidence classification; and

either accept the result, request additional analysis, or route it to a human reviewer.

This creates a useful division of responsibility:

> Orchestration provides reliability.
>
> Deterministic search provides precision.
>
> AI agents provide adaptive reasoning.
>
> Humans provide governance for ambiguous or consequential decisions.

The agent therefore does not replace the underlying search infrastructure. Instead, it acts as a decision-making layer above it, deciding which tools to invoke and what additional evidence is needed before a candidate publication can be classified.

This architecture also provides an important safeguard against "black box" AI. Every agent decision can be associated with the evidence, tools used, search terms considered, confidence score, and final disposition.

The result is not an autonomous AI system making untraceable decisions. It is an auditable, agent-assisted research pipeline in which AI is given autonomy where it adds value while deterministic controls and human review remain in place where they matter most.

In practice, this approach could allow a very small IT team to operate a continuously expanding literature-mining capability without manually managing each publication. The system would process routine cases automatically and concentrate human attention on the relatively small number of ambiguous cases where expert judgment provides the greatest value.

---

6. Store the evidence, not just the answer

Another important design principle is that the system should preserve the evidence behind its conclusions.

The database should not merely contain: Publication X → JAX strain 000664

The conclusion should be auditable.

Instead, the record should retain the evidence:


```
{
  "pmcid": "PMC7572898",
  "pmid": "32853550",
  "doi": "10.1016/j.neuron.2020.07.023",

  "jaxa_stock": "000664",

  "matched_string": "C57BL/6J",

  "identifier_type": "strain_shortname",

  "match_method": "exact",

  "context": "...C57BL/6J mice were used...",

  "interpretation": "Likely experimental use",

  "confidence": 0.94
}
```


This is important for both scientific credibility and business adoption.

A researcher or business analyst should be able to ask:

> **"Why did the system decide that this publication represents JAX usage?"**

and receive the evidence.

That makes the system explainable.

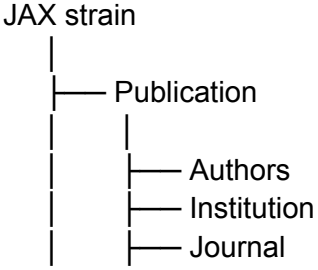
---

7. Build the research intelligence index

The resulting evidence records can be stored in Amazon S3 and indexed in Amazon OpenSearch Serverless.

At this point, the project begins to evolve beyond text search.

The system now contains relationships among:



```
│       └── Research topic
│
├── Disease
│
├── Experimental method
│
└── Other JAX strains
```

That creates a potentially powerful research-intelligence dataset.

Instead of simply asking:

> "Where does JAX stock 000664 occur?"

the organization can eventually ask:

> "What research is being conducted with JAX stock 000664?"

That is a much more valuable question.

From literature search to business intelligence

Once the publication evidence is normalized and indexed, entirely new questions become possible.

For example:

Which JAX strains are appearing most frequently in cancer research?

Which institutions are publishing research involving particular JAX strains?

Which researchers have repeatedly published studies involving JAX animals?

Which JAX strains are associated with emerging areas of biomedical research?

Which publications provide particularly strong evidence of successful use of a JAX strain?

And perhaps most interesting from a business perspective:

Which published research appears to represent a previously unidentified JAX customer or research relationship?

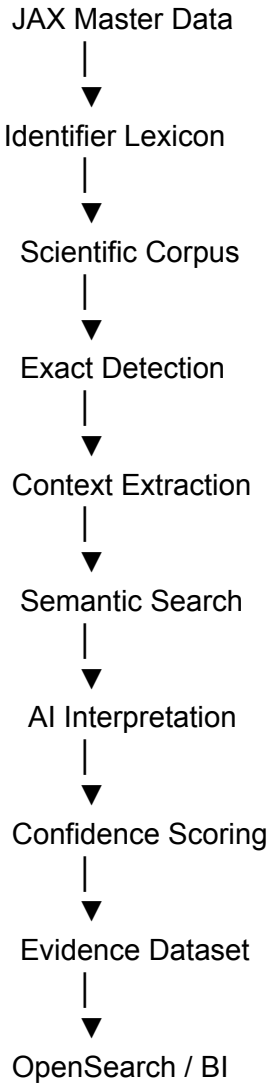
That final question illustrates why I considered this more than a search project.

The ultimate objective was to connect **scientific impact to organizational intelligence**.

The bigger lesson

The most interesting part of this project was not simply the role of the AI agent and LangGraph.

It was the comprehensive architecture created by combining several relatively straightforward capabilities.



The key design principle was to use each technology for the problem it solves best.

S3 provides scalable storage for the corpus and resulting evidence.

Conventional search finds exact identifiers efficiently.

OpenSearch provides keyword and semantic retrieval.

Embeddings help locate conceptually related material.

LLMs interpret ambiguous evidence and normalize relationships (eg. Open AI GPT).

Structured metadata makes the results searchable and auditable.

And humans remain responsible for the decisions that matter.

That last point is particularly important.

An AI-generated match should not automatically become a business fact simply because a language model produced a high confidence score.

Instead, AI should progressively reduce the amount of manual investigation required by a human analyst.

The system might therefore classify results as:

HIGH CONFIDENCE
  │
  └── Automatically accepted

MEDIUM CONFIDENCE
  │
  └── Human review

LOW CONFIDENCE
  │
  └── Retained as a candidate

The objective is not to eliminate human judgment.

It is to ensure that human judgment is spent on the **interesting cases rather than the millions of obvious ones**.

Accuracy would be established through a human-validated gold-standard corpus rather than by assuming that a high AI confidence score represents correctness. A stratified sample of publications would be manually classified for JAX usage, identifier resolution, and evidence strength. The system could then be evaluated independently for recall, precision, and strain-resolution accuracy. After initial validation, human review could focus disproportionately on low-confidence and ambiguous cases, while a smaller random sample of high-confidence results would provide continuing quality assurance. This approach minimizes manual effort while providing an auditable measure of how reliably the automated pipeline discovers and attributes JAX research.

The opportunity hidden in unstructured data

The original business problem sounded deceptively simple:

> "Find publications that mention JAX mice."

But the real problem was considerably more sophisticated.

The organization needed to recognize a research resource despite inconsistent terminology, determine whether the reference represented actual experimental usage, connect different identifiers to a canonical strain, associate that strain with researchers and institutions, and ultimately turn those observations into actionable intelligence.

That is a classic example of where AI becomes most useful when it is **embedded in a broader data architecture**.

The LLM is not the solution by itself.

The value comes from the pipeline around it.

The most important output isn't a chatbot response.

It is a continuously expanding, evidence-backed map connecting:

JAX strains → scientific publications → researchers → institutions → research topics → scientific impact → potential business opportunity.

For an organization whose mission depends on enabling scientific discovery, that connection could be extraordinarily valuable.

The goal of agentic orchestration was not to make the architecture more "AI-like." It was to make the system operationally scalable. A small IT team should be able to launch the process, monitor exceptions, and periodically improve the lexicon and decision rules without manually shepherding millions of documents through the pipeline.

The Jackson Laboratory already had the scientific resources.

The scientific community was already publishing the results.

The data was already available.

The opportunity was to build the intelligence layer that could connect the dots.

REFERENCES:

[NIH NLM NCBI PubMed Central (PMC) Article Datasets - Full-Text Biomedical and Life Sciences Journal Articles on AWS]

[Structure of NLM Pubmed Article Datasets]

[Opportunities and Challenges for ChatGPT and Large Language Models in Biomedicine and Health]